

Relatório — 1.1 Criar o projeto (repo simples)

Escopo: documentar tudo o que fizemos para criar o esqueleto do serviço (NestJS no root), os problemas encontrados, suas **causas-raiz**, as **correções** aplicadas e o **racional** de cada decisão. Útil como material de aula e auditoria.

1) Geração do projeto (Nest CLI) e conflito de arquivos

Ação: `nest new . --package-manager npm --strict`

Sintoma: erro `A merge conflicted on path "/README.md"`.

Causa-raiz (fundamentação): o Nest CLI tenta gerar arquivos padrão (`README.md`, `.gitignore`, `package.json` etc.) e **não sobrescreve** os que já existem. Ao detectar conflito em diretório não vazio, **aborta** para evitar perda de conteúdo.

Correção aplicada (segura): gerar em pasta temporária e **copiar** para o repo, preservando governança.

```
mkdir -p /tmp/nest-skel && cd /tmp/nest-skel
nest new aurora-users --package-manager npm --strict --skip-git --skip-install
rsync -av /tmp/nest-skel/aurora-users/
"/Users/evertoncoimbradearaujo/Documents/GitHub/dsc-2025-2-aurora-platform/"
--exclude '.git' --exclude '.github' --exclude 'README.md'
```

Alternativa (rápida): renomear `README.md` / `.gitignore` antes de `nest new .` e depois mesclar conteúdos.

Estado: branch `feat/nest-skeleton` criada; PR aberto com descrição padronizada.

2) Falha na instalação automática do Nest CLI

Sintoma: `Packages installation failed!` (durante o `nest new`).

Causa-raiz: a etapa automática roda `npm install`; problemas de rede/peer deps/perm podem falhar. Como usamos `--skip-install`, **o esqueleto foi gerado** mesmo assim.

Correção aplicada: instalação **manual** no repo (com logs visíveis) e **versionamento do lockfile**.

```
npm install # gera package-lock.json
```

Racional: a nossa CI usa `npm ci`, que **exige** `package-lock.json`. Sem lockfile, o job "lint" falha em poucos segundos.

3) Lint quebrando localmente (antes de rodar na CI)

3.1 `no-floating-promises` em `src/main.ts`

Causa: chamada a `bootstrap()` sem tratar a Promise.

Fix: tornar explícito que a Promise é ignorada pelo runtime.

```
// src/main.ts
async function bootstrap() { /* ... */ }
void bootstrap();
```

3.2 Regras `no-unsafe-*` no `test/app.e2e-spec.ts`

Causa: uso de `import * as request` + `return request(...)` leva a tipos `any` e "unsafe".

Fix: usar **default import** de `supertest` e **await** (sem `return`).

```
import request from 'supertest';
// ...
it('/ (GET)', async () => {
  await request(app.getHttpServer()).get('/').expect(200).expect('Hello World!');
});
```

Racional: melhora a tipagem e elimina `no-unsafe-return/call/member-access`.

Resultado local: `npm run lint -- --fix` fica sem erros (warning residual aceitável).

4) CI falhando no job "lint" e pulando build/test

Sintoma: no PR, "CI / lint" falha em ~19s; "build" e "test" ficam **skipped**.

Causas combinadas: 1. **Lockfile ausente** (antes da instalação manual) → `npm ci` falha. (Corrigido ao versionar `package-lock.json`.) 2. **Uso de** `npm -ws run ...` no workflow. Em **repo simples** (sem "workspaces" no `package.json`), a flag `-ws` não se aplica e o comando aborta.

Correção aplicada: manter `npm ci` e **trocar os comandos** para respeitar o modo *repo simples* (com detecção futura de workspaces):

```
# .github/workflows/ci.yml (essencial)
- name: Detect workspaces
  id: ws
  run: |
    if grep -q '"workspaces"' package.json; then echo "has_ws=true" >>
$GITHUB_OUTPUT; else echo "has_ws=false" >> $GITHUB_OUTPUT; fi

# Lint (root)
- run: npm run lint --if-present
  if: steps.ws.outputs.has_ws == 'false'
# Lint (workspaces)
- run: npm -ws run lint --if-present
  if: steps.ws.outputs.has_ws == 'true'
# (repetir o par root/workspaces para build e test)
```

Racional: preserva o mesmo workflow quando migrarmos para **monorepo**, sem quebrar o **repo simples** atual.

5) Publicação da branch e PR

Ação: usar GitHub Desktop/CLI para publicar `feat/nest-skeleton` e abrir PR com descrição padronizada.

Pontos de atenção: - Branch Protection exige PR + checks; sem lockfile ou com `-ws` o lint quebra e bloqueia o merge (comportamento desejado para a disciplina). - Após os fixes (itens 2-4), os 3 contexts devem ficar **verdes**.

6) Estado atual

- Esqueleto NestJS **copiado para o root** e versionado.
 - `package-lock.json` versionado (requisito para `npm ci`).
 - Lint **corrigido** em `src/main.ts` e `test/app.e2e-spec.ts`.
 - Workflow **ajustado** para *repo simples* (com detecção de workspaces futura).
 - **PR aberto**; ao rodar os jobs com o workflow atualizado, a expectativa é ter `lint/build/test` **verdes**. Caso algum passo falhe, inspecionar o log do job correspondente.
-

7) Lições aprendidas (para os alunos)

- **Gerar em diretório vazio** ou **copiar de tmp** evita conflitos com governança já existente.
 - `npm ci` $\neq$ `npm install`: CI precisa de lockfile para reprodutibilidade.
 - Lint **não é "cosmético"**: pega classes de erros (promises soltas, typings "unsafe") cedo.
 - Workflows **devem refletir a topologia do repo** (repo simples vs. monorepo). Parametrização evita retrabalho.
-

8) Próximos passos (como estava no roteiro)

1. Ativar **ValidationPipe** e **Swagger** (se ainda não aplicados no `main.ts`) e criar **healthcheck**.
2. Integrar **PostgreSQL + TypeORM** (compose, datasource, `.env.example`).
3. Gerar **migrations** (sem `synchronize:true`).
4. Implementar **CRUD de Users** (email único, `bcrypt`, não retornar hash).
5. Atualizar a **CI** do job `test` para subir Postgres como `service` (quando chegar no TypeORM), e manter os 3 checks como gate.

Checklist para fechar este item 1.1: PR do skeleton com **CI verde** (`lint/build/test`)
+ merge via **Squash and merge** (histórico linear).